



Fabric App Development using Rayfin

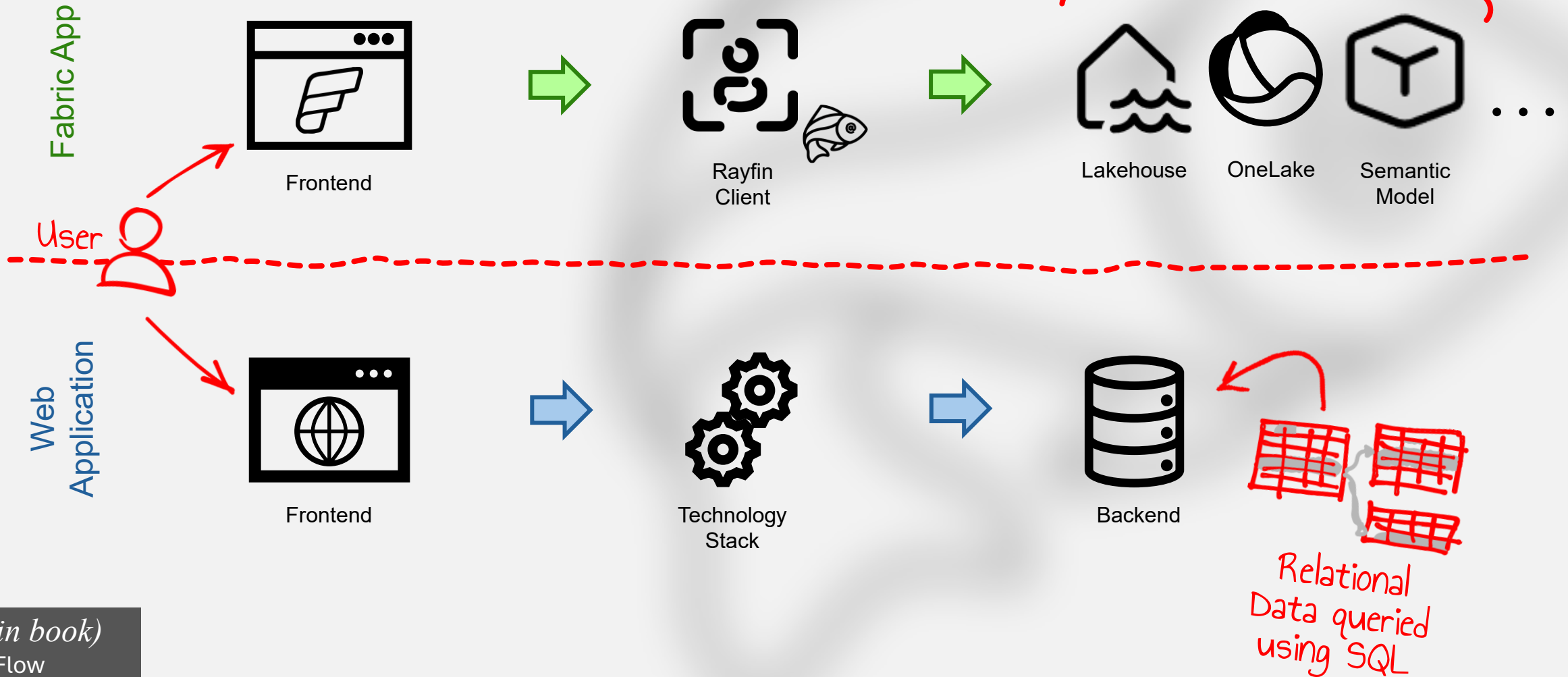
Fabric Apps versus Traditional Application Development

Adnan Hashmi
pkstn.co/Adnan

#Why #What #How

Fabric vs. Traditional Web Apps

Microsoft Fabric
internal data
stored as Parquet



(not in book)
Flow

#Why #What #How

Fabric vs. Traditional Web Apps

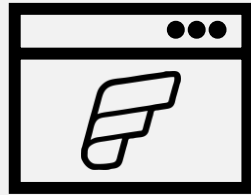


Generated using LLM + Rayfin



Fabric App

Reporting (OLAP)



Frontend



Rayfin
Client



Lakehouse



OneLake



Semantic
Model

...

Predominantly reads;
Writebacks achieved using Translytical
Task flows + Fabric UDFs



(not in book)
Flow



Frontend



Technology
Stack



Backend

Coded manually in IDE



CRUD



Choosing a Development Model

1. Fabric App: Quickly deploy a reporting solution ←
2. Web Application: Achieve read/write with most flexibility ←
3. Hybrid: Requires capabilities from both ←



#Why #What #How

Fabric Apps

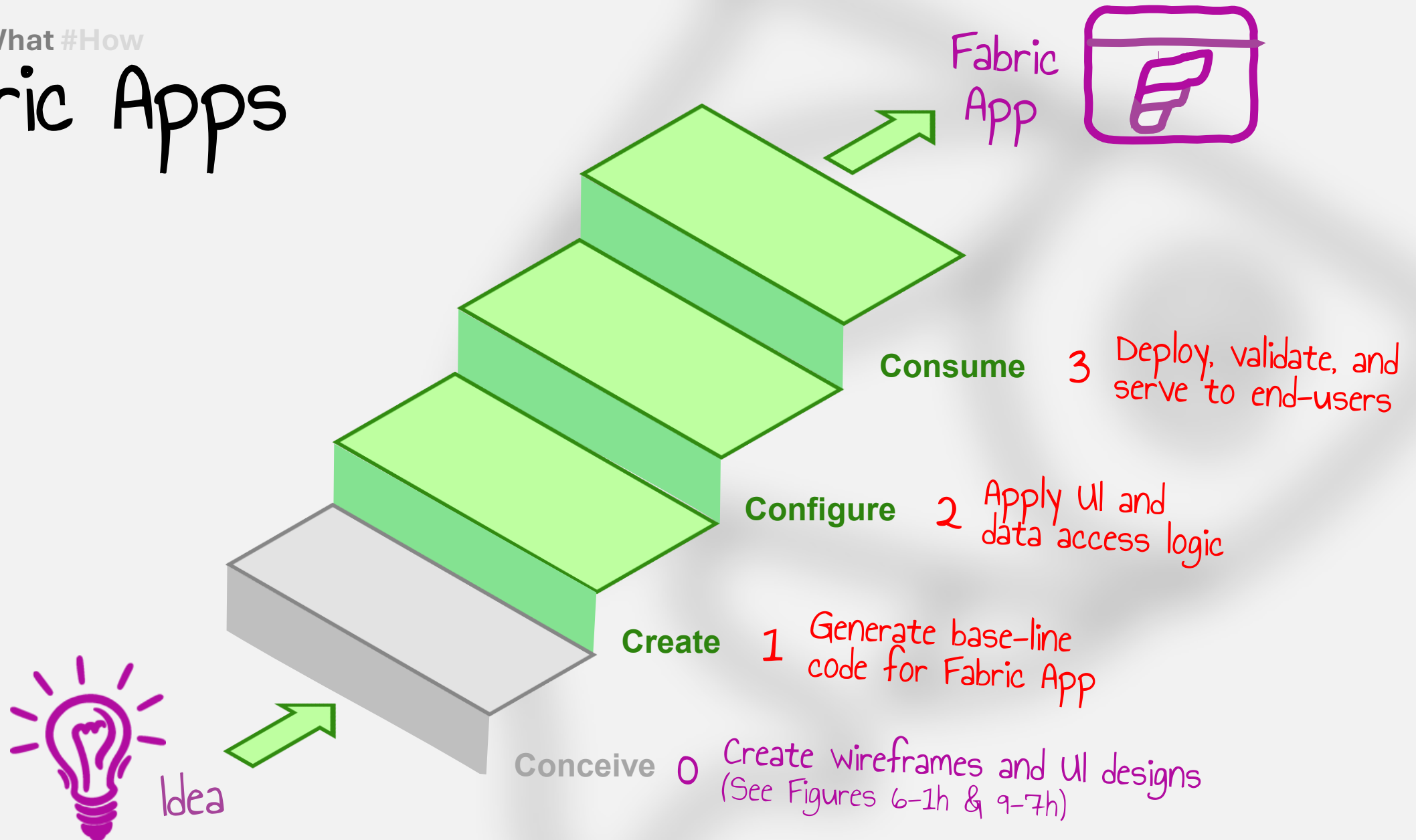
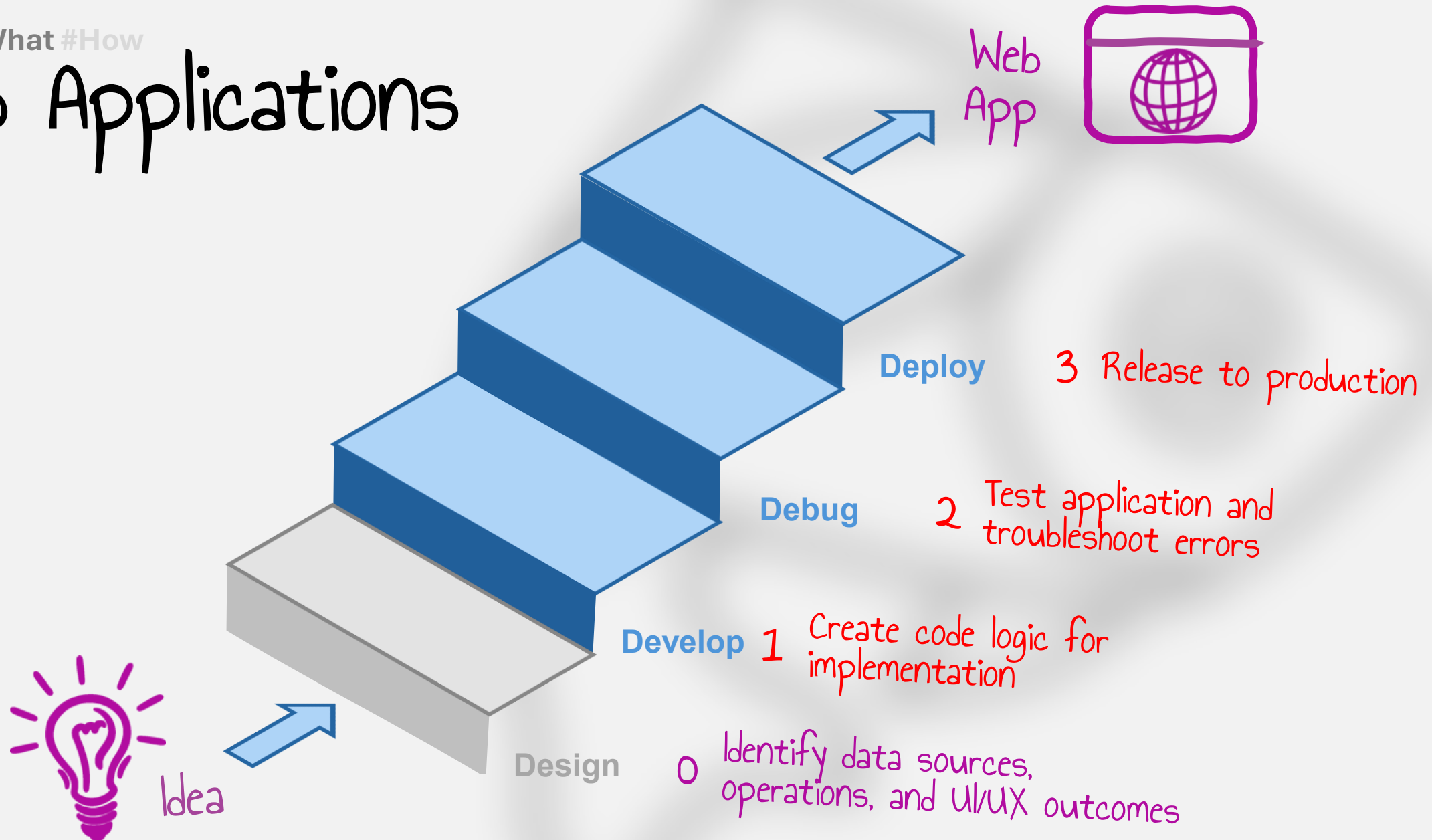


Figure 9-7e
Flow

#Why #What #How

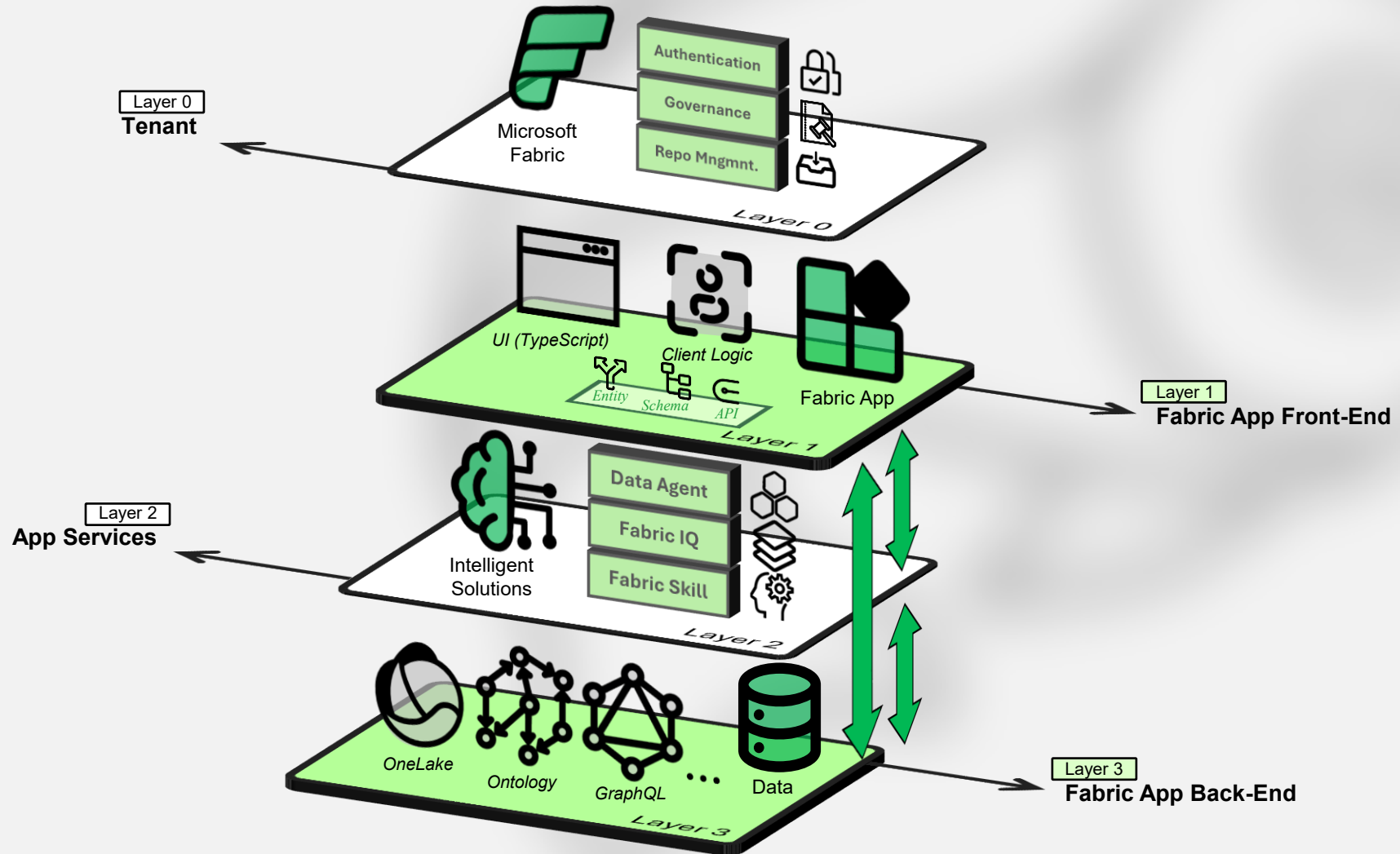
Web Applications



(not in book)
Flow

#Why #What #How

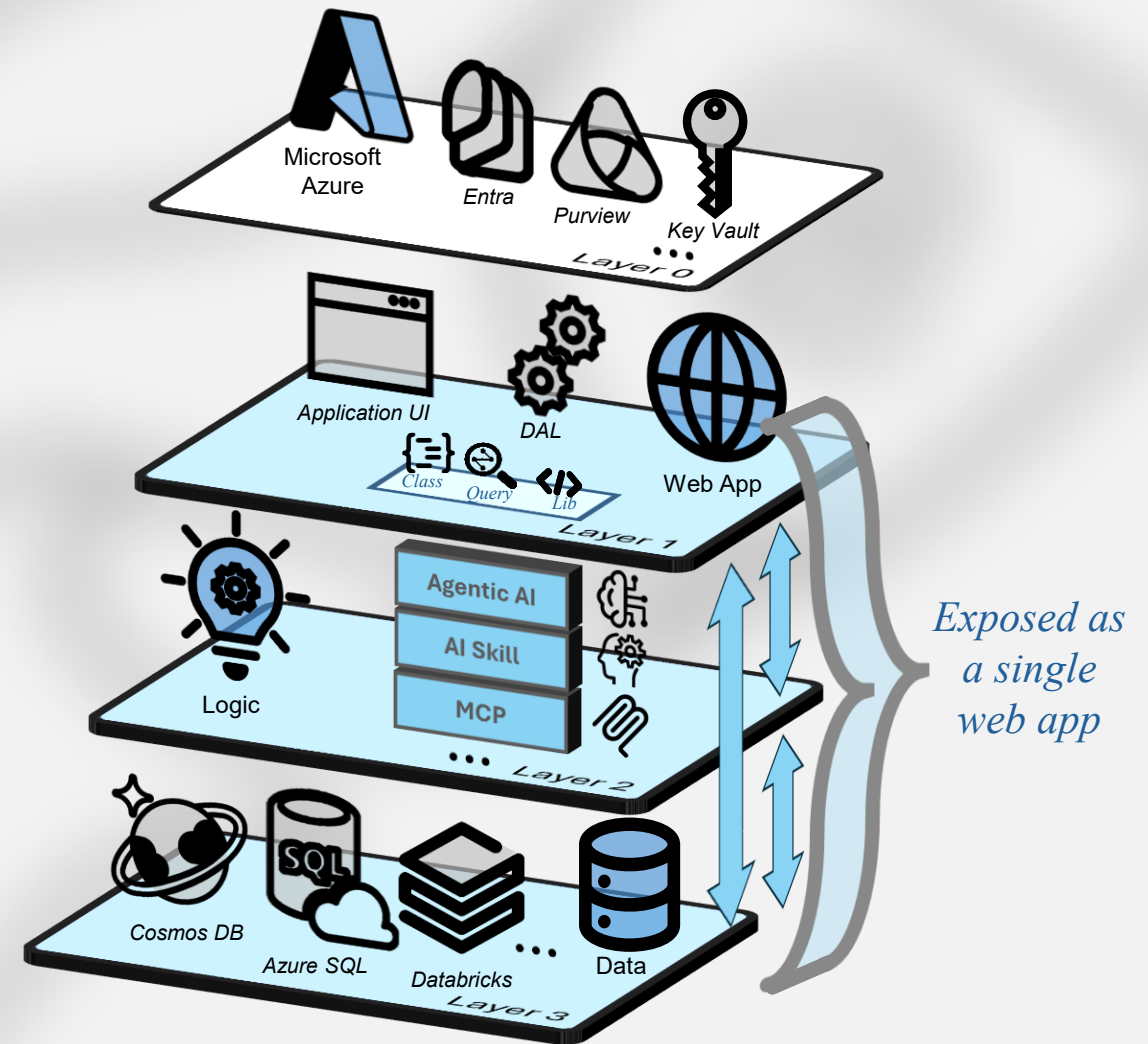
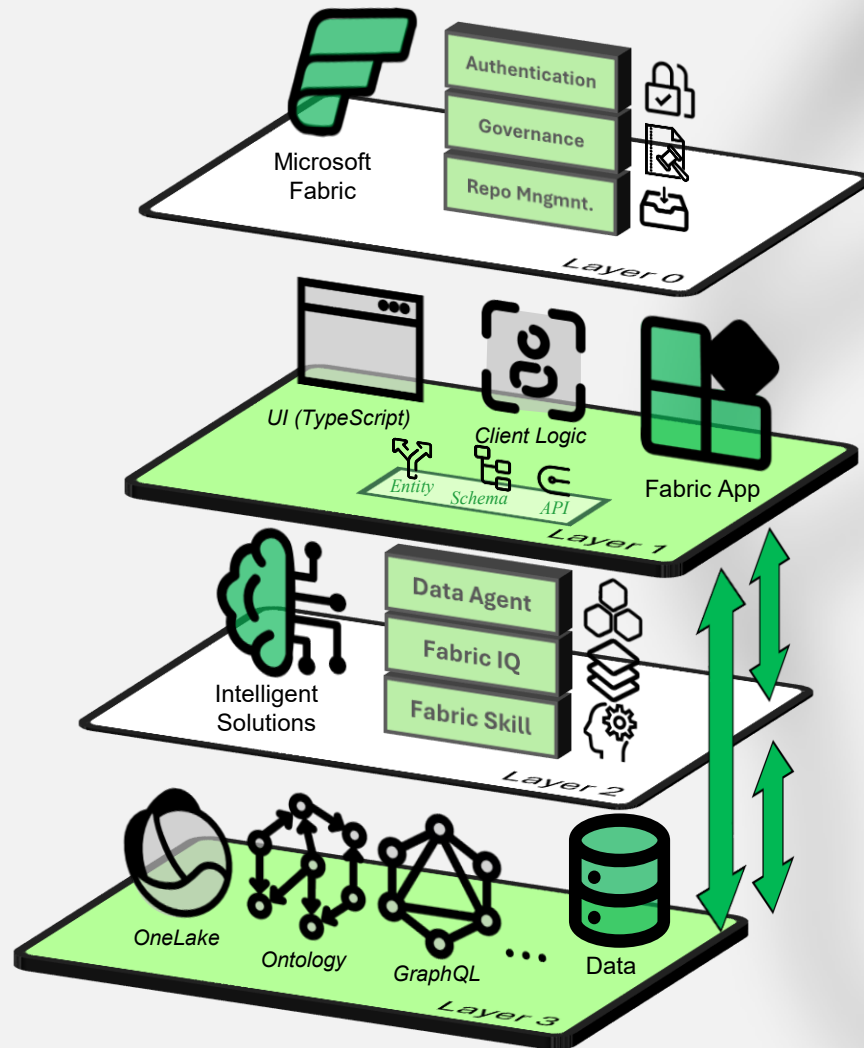
A typical Fabric App



(not in book)
Architecture

#Why #What #How

Fabric App vs. Traditional Web Application



(not in book)
Architecture

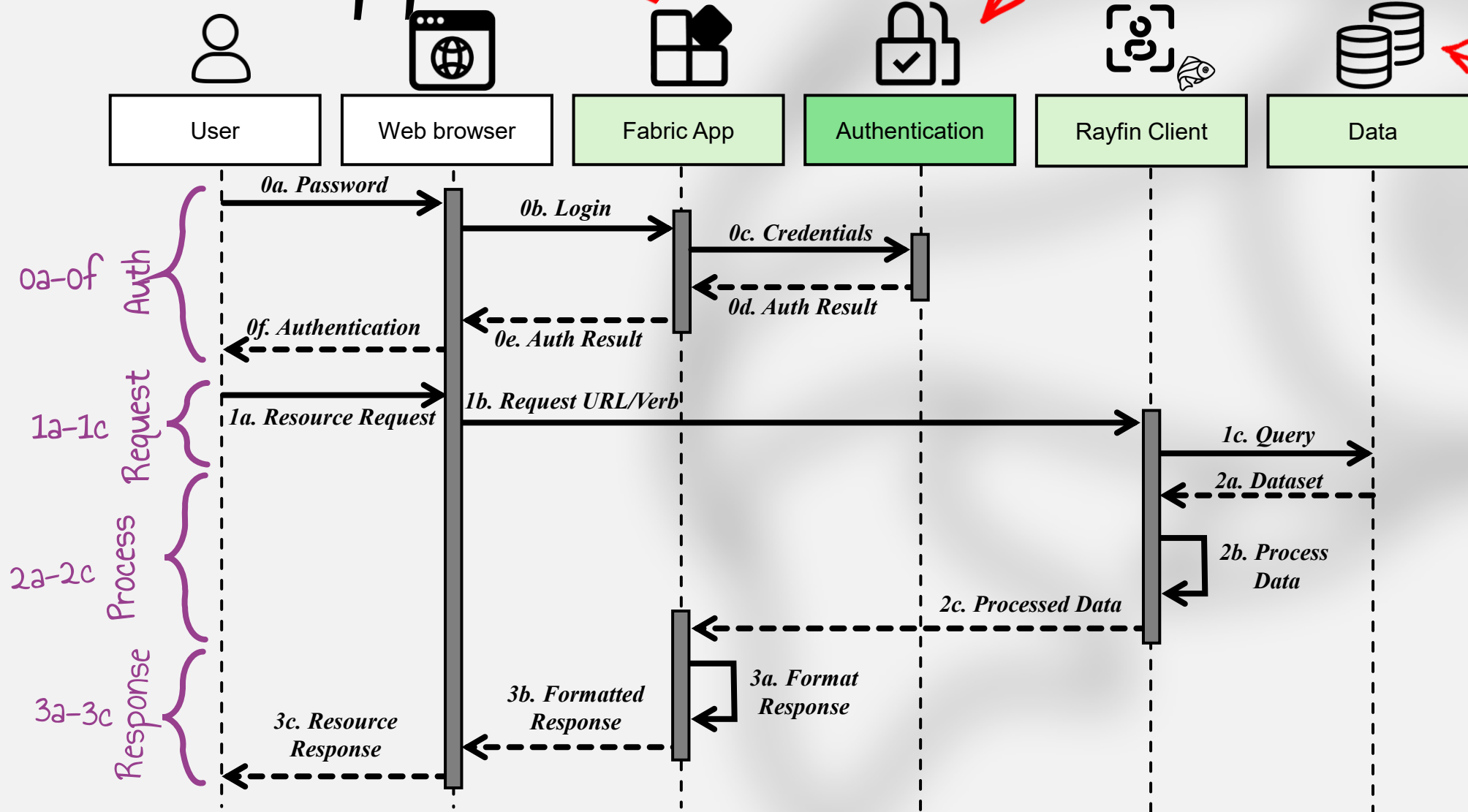
#Why #What #How

Fabric App

Accessed via
endpoint

Provided by
Microsoft Fabric

Includes OneLake,
Semantic Models,
GraphQL, etc.



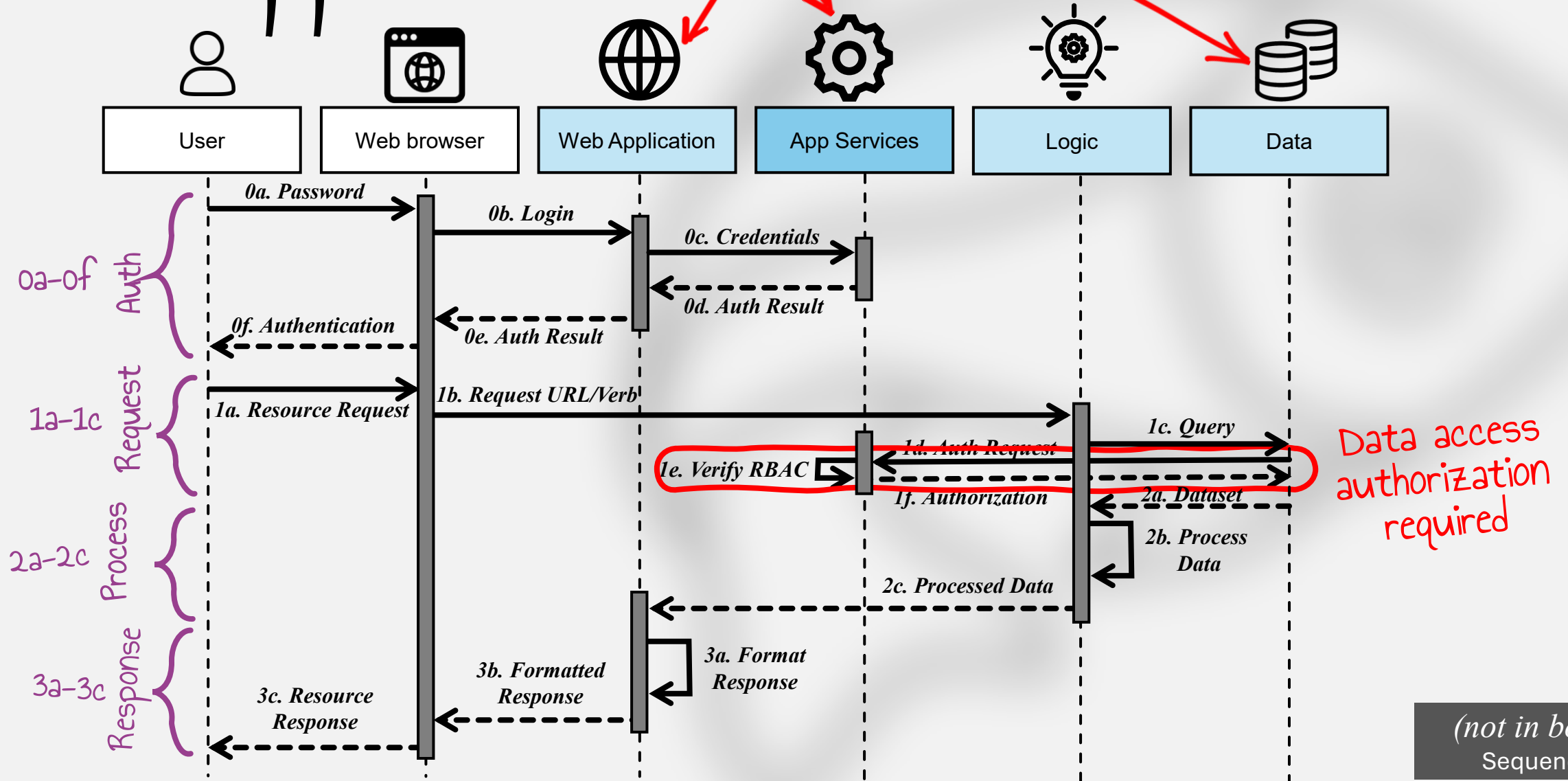
(not in book)
Sequence

#Why #What #How

Web App

Hosted in
Microsoft Azure

Data stored in
Cosmos DB, Azure
SQL, Databricks, etc.



(not in book)
Sequence

#Why #What #How

Recap

1. Choice between Fabric versus traditional web apps
2. Comparison with traditional web applications
3. Architecture & Execution Sequence



Thank You!

